

Lecture 24

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

September 29, 2026

Issues in IR Design

- How much machine dependent

Issues in IR Design

- How much machine dependent
- Expressiveness: how many languages are covered

Issues in IR Design

- How much machine dependent
- Expressiveness: how many languages are covered
- Appropriateness for code optimization

Issues in IR Design

- How much machine dependent
- Expressiveness: how many languages are covered
- Appropriateness for code optimization
- Appropriateness for code generation

Three Address Code

- It is a sequence of statements of the general form `X := Y op Z` where

Three Address Code

- It is a sequence of statements of the general form `X := Y op Z` where
  - `X, Y or Z` are names, constants or compiler generated temporaries

Three Address Code

- It is a sequence of statements of the general form `X := Y op Z` where
  - `X, Y or Z` are names, constants or compiler generated temporaries
  - op stands for any operator such as a fixed- or floating-point arithmetic operator, or a logical operator

Three Address Code

- It is a sequence of statements of the general form `X := Y op Z` where
  - `X, Y or Z` are names, constants or compiler generated temporaries
  - op stands for any operator such as a fixed- or floating-point arithmetic operator, or a logical operator
- Only one operator on the right hand side is allowed

Three Address Code

- It is a sequence of statements of the general form `X := Y op Z` where
  - `X, Y or Z` are names, constants or compiler generated temporaries
  - op stands for any operator such as a fixed- or floating-point arithmetic operator, or a logical operator
- Only one operator on the right hand side is allowed
- Source expression like x + y * z might be translated into

```
t1 := y * z
t2 := x + t1
```

Three Address Code

- It is a sequence of statements of the general form `X := Y op Z` where
  - `X, Y or Z` are names, constants or compiler generated temporaries
  - op stands for any operator such as a fixed- or floating-point arithmetic operator, or a logical operator
- Only one operator on the right hand side is allowed
- Source expression like x + y * z might be translated into

  ```
  t1 := y * z
  t2 := x + t1
  ```

  where t1 and t2 are compiler generated temporary names
- Three address code are a linearized representation of a syntax tree where explicit names correspond to the interior nodes of the graph

Three address instructions

- Assignment
  - x = y op z
  - x = op y
  - x=y

Three address instructions

- Assignment
  - x = y op z
  - x = op y
  - x=y
- Jump
  - goto L
  - if x relop y goto L

Three address instructions

- Assignment
  - x = y op z
  - x = op y
  - x=y
- Jump
  - goto L
  - if x relop y goto L
- Indexed assignment
  - x = y[i]
  - x[i] = y

Three address instructions

- Assignment
  - x = y op z
  - x = op y
  - x=y
- Jump
  - goto L
  - if x relop y goto L
- Indexed assignment
  - x = y[i]
  - x[i] = y
- Function
  - param x
  - call p,n
  - return y

Three address instructions

- Assignment
  - x = y op z
  - x = op y
  - x=y
- Jump
  - goto L
  - if x relop y goto L
- Indexed assignment
  - x = y[i]
  - x[i] = y
- Function
  - param x
  - call p,n
  - return y
- Pointer
  - x = y
  - x = *y
  - *x = y

Other Represenation

- SSA: Single Static Assignment

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language
- Stack machines: P-code

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language
- Stack machines: P-code
- CFG: Control Flow Graph

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language
- Stack machines: P-code
- CFG: Control Flow Graph
- Dominator Trees

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language
- Stack machines: P-code
- CFG: Control Flow Graph
- Dominator Trees
- PDG: Program Dependence Graph

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language
- Stack machines: P-code
- CFG: Control Flow Graph
- Dominator Trees
- PDG: Program Dependence Graph
- VDG: Value Dependence Graph

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language
- Stack machines: P-code
- CFG: Control Flow Graph
- Dominator Trees
- PDG: Program Dependence Graph
- VDG: Value Dependence Graph
- GURRR: Global unified resource requirement representation. Combines PDG with resource requirements

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language
- Stack machines: P-code
- CFG: Control Flow Graph
- Dominator Trees
- PDG: Program Dependence Graph
- VDG: Value Dependence Graph
- GURRR: Global unified resource requirement representation. Combines PDG with resource requirements
- Java intermediate bytecodes

Other Represenation

- SSA: Single Static Assignment
- RTL: Register transfer language
- Stack machines: P-code
- CFG: Control Flow Graph
- Dominator Trees
- PDG: Program Dependence Graph
- VDG: Value Dependence Graph
- GURRR: Global unified resource requirement representation. Combines PDG with resource requirements
- Java intermediate bytecodes
- The list goes on $\cdots\cdots$

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names
- Symbol table must have mechanism to:

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names
- Symbol table must have mechanism to:
  - add new entries

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names
- Symbol table must have mechanism to:
  - add new entries
  - find existing information efficiently

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names
- Symbol table must have mechanism to:
  - add new entries
  - find existing information efficiently
- Two common mechanisms:

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names
- Symbol table must have mechanism to:
  - add new entries
  - find existing information efficiently
- Two common mechanisms:
  - linear lists, simple to implement, poor performance

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names
- Symbol table must have mechanism to:
  - add new entries
  - find existing information efficiently
- Two common mechanisms:
  - linear lists, simple to implement, poor performance
  - hash tables, good performance, need good hash function

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names
- Symbol table must have mechanism to:
  - add new entries
  - find existing information efficiently
- Two common mechanisms:
  - linear lists, simple to implement, poor performance
  - hash tables, good performance, need good hash function
- Compiler should be able to grow symbol table dynamically

Symbol table

- Compiler uses symbol table to keep track of scope and binding information about names
- Symbol table must have mechanism to:
  - add new entries
  - find existing information efficiently
- Two common mechanisms:
  - linear lists, simple to implement, poor performance
  - hash tables, good performance, need good hash function
- Compiler should be able to grow symbol table dynamically
- If size is fixed, it must be large enough for the largest program

Symbol Attributes and Symbol Table Entries

- Symbols have associated attributes

Symbol Attributes and Symbol Table Entries

- Symbols have associated attributes
- Typical attributes are name, type, scope, size, addressing mode etc

Symbol Attributes and Symbol Table Entries

- Symbols have associated attributes
- Typical attributes are name, type, scope, size, addressing mode etc
- A symbol table entry collects together attributes such that they can be easily set and retrieved

Symbol Attributes and Symbol Table Entries

- Symbols have associated attributes
- Typical attributes are name, type, scope, size, addressing mode etc
- A symbol table entry collects together attributes such that they can be easily set and retrieved
- Example of typical names in symbol table

Name	Type
name	string
size	int
type	enum

Symbol Attributes and Symbol Table Entries

- Symbols have associated attributes
- Typical attributes are name, type, scope, size, addressing mode etc
- A symbol table entry collects together attributes such that they can be easily set and retrieved
- Example of typical names in symbol table

Name	Type
name	string
size	int
type	enum

- A major consideration in designing a symbol table is that insertion and retrieval should be as fast as possible

Symbol Table Entries

- Each entry for a declaration of a name

Symbol Table Entries

- Each entry for a declaration of a name
- Format need not be uniform because information depends upon the usage of the name

Symbol Table Entries

- Each entry for a declaration of a name
- Format need not be uniform because information depends upon the usage of the name
- Each entry is a record consisting of consecutive words

Symbol Table Entries

- Each entry for a declaration of a name
- Format need not be uniform because information depends upon the usage of the name
- Each entry is a record consisting of consecutive words
- To keep records uniform some entries may be outside the symbol table

Symbol Table Entries

- Each entry for a declaration of a name
- Format need not be uniform because information depends upon the usage of the name
- Each entry is a record consisting of consecutive words
- To keep records uniform some entries may be outside the symbol table
- Information is entered into symbol table at various times

Symbol Table Entries

- Each entry for a declaration of a name
- Format need not be uniform because information depends upon the usage of the name
- Each entry is a record consisting of consecutive words
- To keep records uniform some entries may be outside the symbol table
- Information is entered into symbol table at various times
  - Keywords are entered initially

Symbol Table Entries

- Each entry for a declaration of a name
- Format need not be uniform because information depends upon the usage of the name
- Each entry is a record consisting of consecutive words
- To keep records uniform some entries may be outside the symbol table
- Information is entered into symbol table at various times
  - Keywords are entered initially
  - Identifier lexemes are entered by lexical analyzer

Symbol Table Entries

- Each entry for a declaration of a name
- Format need not be uniform because information depends upon the usage of the name
- Each entry is a record consisting of consecutive words
- To keep records uniform some entries may be outside the symbol table
- Information is entered into symbol table at various times
  - Keywords are entered initially
  - Identifier lexemes are entered by lexical analyzer
- Attribute values are filled in as information is available

Data Structures

- List data structure

Data Structures

- List data structure
  - simplest to implement

Data Structures

- List data structure
  - simplest to implement
  - uses a single array to store names and information

Data Structures

- List data structure
  - simplest to implement
  - uses a single array to store names and information
  - search for a name is linear

Data Structures

- List data structure
  - simplest to implement
  - uses a single array to store names and information
  - search for a name is linear
  - entry and lookup are independent operations

Data Structures

- List data structure
  - simplest to implement
  - uses a single array to store names and information
  - search for a name is linear
  - entry and lookup are independent operations
  - cost of entry and search operations are very high and lot of time goes into bookkeeping.

Data Structures

- List data structure
  - simplest to implement
  - uses a single array to store names and information
  - search for a name is linear
  - entry and lookup are independent operations
  - cost of entry and search operations are very high and lot of time goes into bookkeeping.
- Hash table

Data Structures

- List data structure
  - simplest to implement
  - uses a single array to store names and information
  - search for a name is linear
  - entry and lookup are independent operations
  - cost of entry and search operations are very high and lot of time goes into bookkeeping.
- Hash table
  - The advantages are obvious

Representing Scope Information

- Entries are declarations of names.

Representing Scope Information

- Entries are declarations of names.
- When a lookup is done, an entry for an appropriate declaration must be returned.

Representing Scope Information

- Entries are declarations of names.
- When a lookup is done, an entry for an appropriate declaration must be returned.
- Scope rules determine which entry is appropriate

Representing Scope Information

- Entries are declarations of names.
- When a lookup is done, an entry for an appropriate declaration must be returned.
- Scope rules determine which entry is appropriate
- Maintain a separate table for each scope

Representing Scope Information

- Entries are declarations of names.
- When a lookup is done, an entry for an appropriate declaration must be returned.
- Scope rules determine which entry is appropriate
- Maintain a separate table for each scope
- Information about non-local is found by scanning the symbol table for the enclosing procedures

Representing Scope Information

- Entries are declarations of names.
- When a lookup is done, an entry for an appropriate declaration must be returned.
- Scope rules determine which entry is appropriate
- Maintain a separate table for each scope
- Information about non-local is found by scanning the symbol table for the enclosing procedures
- Names are entered in the symbol table in the order they occur

Representing Scope Information

- Entries are declarations of names.
- When a lookup is done, an entry for an appropriate declaration must be returned.
- Scope rules determine which entry is appropriate
- Maintain a separate table for each scope
- Information about non-local is found by scanning the symbol table for the enclosing procedures
- Names are entered in the symbol table in the order they occur
- Most closely nested rule can be created in terms of the following operations:

Representing Scope Information

- Entries are declarations of names.
- When a lookup is done, an entry for an appropriate declaration must be returned.
- Scope rules determine which entry is appropriate
- Maintain a separate table for each scope
- Information about non-local is found by scanning the symbol table for the enclosing procedures
- Names are entered in the symbol table in the order they occur
- Most closely nested rule can be created in terms of the following operations:
  - lookup: find the most recently created entry
  - insert: make a new entry
  - delete: remove the most recently created entry

Declarations

- For each name create symbol table entry with information like type and relative address
- $P \rightarrow \{offset = 0\} \quad D$
- $D \rightarrow D; D$

Declarations

- For each name create symbol table entry with information like type and relative address
- $P \rightarrow \{offset = 0\} \quad D$
- $D \rightarrow D; D$
- $D \rightarrow id : T$

Declarations

- For each name create symbol table entry with information like type and relative address
- $P \rightarrow \{offset = 0\} \quad D$
- $D \rightarrow D; D$
- $D \rightarrow id : T \quad enter(id.name, T.type, offset);$
  $offset = offset + T.width$

Declarations

- For each name create symbol table entry with information like type and relative address
- $P \rightarrow \{offset = 0\} \quad D$
- $D \rightarrow D; D$
- $D \rightarrow id : T \quad enter(id.name, T.type, offset);$
  $offset = offset + T.width$
- $T \rightarrow integer$

Declarations

- For each name create symbol table entry with information like type and relative address
- $P \rightarrow \{offset = 0\} \quad D$
- $D \rightarrow D; D$
- $D \rightarrow id : T \quad enter(id.name, T.type, offset);$
  $offset = offset + T.width$
- $T \rightarrow integer \quad T.type = integer; T.width = 4$
- $T \rightarrow real$

Declarations

- For each name create symbol table entry with information like type and relative address
- $P \rightarrow \{offset = 0\} \quad D$
- $D \rightarrow D; D$
- $D \rightarrow id : T \quad enter(id.name, T.type, offset);$
  $offset = offset + T.width$
- $T \rightarrow integer \quad T.type = integer; T.width = 4$
- $T \rightarrow real \quad T.type = real; T.width = 8$
- $T \rightarrow array[num]$ of T_1

Declarations

- For each name create symbol table entry with information like type and relative address
- $P \rightarrow \{offset = 0\} \quad D$
- $D \rightarrow D; D$
- $D \rightarrow id : T \quad enter(id.name, T.type, offset);$
  $offset = offset + T.width$
- $T \rightarrow integer \quad T.type = integer; T.width = 4$
- $T \rightarrow real \quad T.type = real; T.width = 8$
- $T \rightarrow array[num]$ of T_1
  $T.type = array(num.val, T_1.type)$
  $T.width = num.val \times T_1.width$
- $T \rightarrow *T_1$

Declarations

- For each name create symbol table entry with information like type and relative address
- $P \rightarrow \{offset = 0\} \quad D$
- $D \rightarrow D; D$
- $D \rightarrow id : T \quad enter(id.name, T.type, offset);$
  $offset = offset + T.width$
- $T \rightarrow integer \quad T.type = integer; T.width = 4$
- $T \rightarrow real \quad T.type = real; T.width = 8$
- $T \rightarrow array[num]$ of T_1
  $T.type = array(num.val, T_1.type)$
  $T.width = num.val \times T_1.width$
- $T \rightarrow *T_1 \;\; T.type = pointer(T_1.type)$
  $T.width = 4$